FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Authors: Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré

ABSTRACT

Transformers are slow and memory-hungry on long sequences, since the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods have attempted to address this compute and memory bottleneck by trading off model quality to achieve speedup, but often fail to achieve real-wallclock speedup. We argue that a missing principle is making attention algorithms IO-aware?accounting for reads and writes between levels of GPU memory (such as High Bandwidth Memory HBM and fast on-chip SRAM). We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU HBM and SRAM. We analyze the IO complexity of FlashAttention, proving that it requires fewer HBM accesses than standard attention. FlashAttention trains Transformers 2-4x faster than standard PyTorch attention, scales linearly in sequence length memory footprint, and enables training on sequences up to 64k tokens.

1. Introduction & The IO Bottleneck Principle

Standard attention implementations in PyTorch materialize the full intermediate N x N attention matrix S = Q * K^T and attention probability matrix P = softmax(S) in GPU High Bandwidth Memory (HBM). Modern GPUs (such as NVIDIA A100) are memory-bandwidth-bound rather than compute-bound for attention: SRAM bandwidth is ~19 TB/s while HBM bandwidth is only ~1.5 TB/s. Materializing large N x N matrices in slow HBM dominates execution time, leading to memory-bound latency.

2. Hardware Foundations: The GPU Memory Hierarchy

To accelerate attention, algorithms must minimize reads and writes between slow HBM (40-80 GB) and small, fast on-chip SRAM (192 KB per Streaming Multiprocessor). Theoretical FLOP reductions that ignore memory IO (such as sparse or low-rank attention) often yield zero practical wall-clock speedup due to memory transfer overhead.

3. FlashAttention Algorithm: Tiling and Online Softmax

FlashAttention computes exact attention using two synergistic techniques:
1. Tiling: We partition inputs Q, K, V into blocks that fit within SRAM. We evaluate softmax iteratively across blocks using the online softmax formulation, maintaining running normalization statistics (m_i, l_i) without materializing the full N x N matrix in HBM.
2. Recomputation: In the backward pass, rather than reading saved N x N attention matrices from slow HBM, FlashAttention recomputes attention blocks directly in fast SRAM using stored forward normalization statistics. This converts slow memory IO into fast arithmetic compute.

4. IO Complexity & End-to-End Speedups

FlashAttention requires O(N^2 * d^2 / M) HBM accesses where M is SRAM capacity, compared to standard attention which requires Theta(N * d + N^2) HBM accesses?a 4x to 9x reduction in HBM traffic. Empirically, FlashAttention trains BERT-large 2.4x faster, GPT-2 3x faster, and solves the Long Range Arena Path-X 16k benchmark with 86.4% accuracy where standard attention runs out of memory.

5. Hardware Constraints & Algorithmic Limitations

FlashAttention kernels require low-level CUDA and warp-level primitives tuned for specific GPU microarchitectures (SRAM sizes, warp shuffles). Execution on non-NVIDIA accelerators or CPU clusters requires ground-up kernel rewriting. Furthermore, for sequence lengths > 100k, IO-aware exact attention still scales as O(N^2) total operations.

6. Future Directions

Key frontiers include hardware-aware multi-query attention, extending IO awareness to structured state space models (Mamba), and mixed-precision FP8 SRAM tiling kernels.